

You can also decide to limit your backup scope, either by schema or data, to a few tables by means of the `-t` command-line flag or, on the other hand, to exclude some tables by means of the `-T` parameter. For example, if we want to back up only the `users` table and `users_pk_seq` sequence, we can do the following:

```
$ pg_dump -f users.sql -t forum.users -t forum.user_pk_seq forumdb
```

The created `users.sql` file will contain only enough data to recreate the user-related stuff and nothing more. On the other hand, if we want to exclude the `users` table from the backup, we can do something similar to the following:

```
$ pg_dump -f users.sql -T forum.users -T forum.user_pk_seq forumdb
```

Of course, you can mix and match any option in a way that makes sense to you and, more importantly, allows you to restore exactly what you need. As an example, if you want to get all the data contained in the `posts` table and the table structure itself, you can do the following:

```
$ pg_dump -f posts.sql -t forum.posts -a -v forumdb
...
pg_dump: warning: there are circular foreign-key constraints on this
table:
pg_dump:   posts
pg_dump: You might not be able to restore the dump without using
--disable-triggers or temporarily dropping the constraints.
pg_dump: Consider using a full dump instead of a --data-only dump to avoid
this problem.
```

`pg_dump` is smart enough to see that the `posts` table has different dependencies and foreign keys, so it warns you that your dump won't be able to restore all the content of the `posts` table. It is up to you to manage such dependencies in the correct way, since you asked `pg_dump` to not perform a full backup (which is always complete and consistent).

Compression

`pg_dump` provides a special command-line option, `-Z`, which accepts an integer from 0 to 9 to indicate a compression level for the backup to produce. The level 0 means no compression at all, while 9 is the highest compression available.

In order to demonstrate how compression works, assume we take two backups of the same database, the first uncompressed and the second compressed (note how command-line options and filenames change):

```
$ pg_dump -f backup_forumdb_uncompressed.sql forumdb
$ pg_dump -f backup_forumdb_compressed.sql.gz -Z 9 forumdb
$ ls -ls backup_forumdb*
4 backup_forumdb_compressed.sql.gz
12 backup_forumdb_uncompressed.sql
```

As you can see, the compressed file takes a third of the space of the plain backup, but this time, the file is not directly editable, since it has been stored in a binary compressed format.

Compression can be applied to plain text (i.e., SQL) dumps, and to directory format, but not to the tar output format.

Dump formats and `pg_restore`

In the previous sections, you only saw the plain SQL format for backups and restores, but `pg_dump` offers more complex and smart formats. All formats except plain SQL must be used with `pg_restore` for restoration and, therefore, are not suitable for manual editing.

Backup formats are specified by the `-F` command-line argument to `pg_dump`, which allows for one of the following values:

- `c` (custom) is the PostgreSQL-specific format within a single file archive.
- `d` (directory) is a PostgreSQL-specific format that's compressed, where every object is split across different files within a directory.
- `t` (tar) is a `.tar` uncompressed format that, once extracted, results in the same layout as the one provided by the directory format.

Let's start with the first format: the custom single-file format. The command to back up a database resembles the one used for the plain SQL format, where you have to specify the output file, but this time, the file is not a plain text one:

```
$ pg_dump -Fc --create -f backup_forumdb.backup forumdb
```

The produced output file is smaller in size than the plain SQL one and can't be edited as text because it is binary. Many of the `pg_dump` command-line arguments apply the same to the custom formats, while others do not make sense at all.

In any case, `pg_dump` is smart enough to know what to take into account and what to discard, so the following command lines will produce the same backup shown in the preceding example:

```
$ pg_dump -Fc --create --inserts -f backup_forumdb.backup forumdb
$ pg_dump -Fc --create --column-inserts -f backup_forumdb.backup forumdb
```

Clearly, the `--column-inserts` and `--inserts` command-line flags do not make any sense in this kind of backup, since no text file (and, therefore, no SQL statement) will be produced.

Once you have the custom backup, how can you restore the database content? Remember that custom backup formats require `pg_restore` to be used for a successful restoration. As we did previously, let's destroy our database again and restore it by means of `pg_restore`:

```
$ psql -c 'DROP DATABASE forumdb';
DROP DATABASE
$ pg_restore -C -d postgres backup_forumdb.backup
```

`pg_restore` runs silently and restores the specified database. The `-C` option indicates that `pg_restore` will recreate the database before restoring objects into it. The `-d` option tells the program to connect to the postgres database first, issue a `CREATE DATABASE`, and then connect to the newly created database to continue the restore, similar to what the plain backup format did. Clearly, `pg_restore` requires a mandatory file to operate on – that is, the last argument specified on the command line.

It is interesting to note that `pg_restore` can produce a list of SQL statements that will be executed without actually executing them. The `-f` command-line option does this, allowing you to store plain SQL in a file or inspect it before proceeding any further with the restoration:

```
$ pg_restore backup_forumdb.backup -f restore.sql
$ less restore.sql
--
-- PostgreSQL database dump
--

CREATE DATABASE forumdb WITH TEMPLATE = template0 ENCODING = 'UTF8'
LOCALE_PROVIDER = libc LOCALE = 'en_US.utf8';

ALTER DATABASE forumdb OWNER TO forum;
```

```
\connect forumdb
```

```
...
```

As you can see, the content of the `restore.sql` file is plain SQL, similar to the output of a plain dump by means of `pg_dump`. This means that if you use `pg_restore`, you can always get an editable and human-readable list of SQL statements out of a backup.

Another output format for `pg_dump` is the directory one, specified by means of the `-Fd` command-line flag. In this format, `pg_dump` creates a set of compressed files in a directory on disk; in this case, the `-f` command-line argument specifies the name of a directory instead of a single file. As an example, let's do a backup in a backup folder:

```
$ pg_dump -Fd -f backup.d forumdb
$ ls -ls backup.d/
total 40
4 3368.dat.gz
4 3370.dat.gz
4 3371.dat.gz
4 3372.dat.gz
4 3373.dat.gz
4 3375.dat.gz
4 3377.dat.gz
12 toc.dat
```

The directory is created, if needed, and every database object is placed in a single compressed file. The `toc.dat` file represents a *ToC*, an index that tells `pg_restore` where to find any piece of data inside the directory. The following example shows you how to destroy and restore the database by means of a backup in the directory format:

```
$ psql -c "DROP DATABASE forumdb;"
DROP DATABASE
$ pg_restore -C -d postgres backup.d
```

The directory backup format is useful when the database grows in size, since it can become a problem to store a single huge file that could overtake the filesystem's limitations.

The very last `pg_dump` format is the `.tar` one, which can be obtained by means of the `-Ft` command-line flag. The result is the creation of a `tar(1)` uncompressed archive that contains the same directory structure that we created in the previous example, but where every file is not compressed:

```
$ pg_dump -Ft -f backup_forumdb.tar forumdb
$ tar -tf backup_forumdb.tar
toc.dat
3368.dat
3370.dat
3371.dat
3372.dat
3373.dat
3375.dat
3377.dat
```

Next, we will look at running a selective restore, which will help you choose which elements of a database you want to restore.

Performing a selective restore

When performing a plain SQL database dump, you are allowed to manually edit the result, since it is plain text, and selectively remove parts you don't want to restore. With custom formats and `pg_restore`, you can do the very same thing, but you need to perform a few steps to do so.

First of all, you can always inspect the content of a binary dump by means of `pg_restore` and its `--list` option, which prints the index (**Table of Contents** or **ToC** for short) to the screen. You need to specify, after the `--list` option, either the single file or directory that contains the backup to get the TOC printed:

```
$ pg_restore --list backup.d
;
; Archive created at 2023-09-29 15:35:12 UTC
;   dbname: forumdb
;   TOC Entries: 39
;   Compression: -1
;   Dump Version: 1.14-0
;   Format: DIRECTORY
;
;
```

```

; Selected TOC Entries:
;
6; 2615 16653 SCHEMA - forum forum
215; 1259 16654 TABLE forum categories forum
216; 1259 16659 SEQUENCE forum categories_pk_seq forum
217; 1259 16660 TABLE forum delete_posts forum
218; 1259 16665 TABLE forum j_posts_tags forum
219; 1259 16668 TABLE forum new_categories forum
220; 1259 16673 TABLE forum posts forum
221; 1259 16682 SEQUENCE forum posts_pk_seq forum
222; 1259 16683 TABLE forum tags forum
223; 1259 16688 SEQUENCE forum tags_pk_seq forum
224; 1259 16689 TABLE forum users forum
225; 1259 16694 SEQUENCE forum users_pk_seq forum
3368; 0 16654 TABLE DATA forum categories forum
3370; 0 16660 TABLE DATA forum delete_posts forum
3371; 0 16665 TABLE DATA forum j_posts_tags forum
3372; 0 16668 TABLE DATA forum new_categories forum
3373; 0 16673 TABLE DATA forum posts forum
3375; 0 16683 TABLE DATA forum tags forum
3377; 0 16689 TABLE DATA forum users forum
3385; 0 0 SEQUENCE SET forum categories_pk_seq forum

...

```

Lines beginning with a semicolon are comments, and as you can see, the first few lines that are printed out are a banner that describes the content of the backup, the date the backup was taken, the format (in this example, “directory”), and how many entries (objects) are in the backup.

Every line that is not a comment represents a database object or a single action that the restore process will perform. As an example, take a look at the following line:

```
222; 1259 16683 TABLE forum tags forum
```

This indicates that the tags table will be restored by the user forum within the forum schema.

The following line means that the same table will be filled with the data:

```
3373; 0 16673 TABLE DATA forum posts forum
```

Thanks to this ToC, you can take control of the restoration process. In fact, if you move or delete lines from the ToC, you can instruct `pg_restore` to change its execution. As an example, first, let's store the ToC in a text file:

```
$ pg_restore --list backup.d > custom_toc.txt
```

Now, edit the `custom_toc.txt` file with your favorite editor and comment out the mentioned part as follows, by placing a semicolon as the first character of the line or by removing the lines that fill the tags table and the related join table:

```
;3373; 0 16673 TABLE DATA forum posts forum
;3371; 0 16665 TABLE DATA forum j_posts_tags forum
```

Now, save the `custom_toc.txt` file. With that, it is possible to restore the database by means of `pg_restore`, but you have to instruct the program to follow your own ToC and not the full and unmodified one that ships with the backup itself. To this aim, `pg_restore` allows the `-L` flag to be specified with the ToC to use:

```
$ psql -c 'DROP DATABASE forumdb;'
DROP DATABASE

$ pg_restore -C -d postgres -L custom_toc.txt backup.d
$ psql -c 'SELECT count(*) FROM forum.tags;' forumdb
count
-----
     0
(1 row)
```

As you can see, the table has been created, but it is empty. This demonstrates how you can drive the restoration of a backup to selectively rearrange the objects to restore.

It is also possible to rearrange lines to make some objects get restored before others, but this is much more complicated, particularly when cross-references and dependencies between objects exist. Anyway, this is an incredibly flexible way to selectively decide what to restore and, moreover, create a different ToC to restore the same format backup in different working sets.

Dumping a whole cluster

`pg_dumpall` is the tool to use to dump a full cluster. In short, `pg_dumpall` loops over all the databases available in the cluster and performs a single `pg_dump` on each, and then it dumps the specific objects that are at a cluster level, such as roles.

`pg_dumpall` works similarly to `pg_dump`, so pretty much all the concepts and options you have seen in the previous sections apply to `pg_dumpall` too. If you don't specify any output format and file, `pg_dumpall` prints all the required SQL statements on the standard output. Assuming you want to store the whole database content in a single SQL file, the following command line provides a full backup:

```
$ pg_dumpall -f cluster.sql
```

The file can become large quickly, and this time, it begins by creating all the required roles:

```
$ less cluster.sql
...
CREATE ROLE book_authors;
ALTER ROLE book_authors WITH NOSUPERUSER INHERIT NOCREATEROLE NOCREATEDB
NOLOGIN NOREPLICATION NOBYPASSRLS;
CREATE ROLE enrico;
ALTER ROLE enrico WITH NOSUPERUSER INHERIT NOCREATEROLE NOCREATEDB LOGIN
NOREPLICATION NOBYPASSRLS PASSWORD 'SCRAM-SHA-256$4096:PiAJvQ9sn/Tcr1cfhJF
isQ==$+gEEKa0oYVLPYNS1o4z04Jng0qAwajBe3DHirEkJT40=:gek2heW0JT+G+8dJa
zqtn4x3Wl5zYY0DyyyKed7pvXY=';
CREATE ROLE forum;
...
```

It then continues by restoring every single database, including `template1`. Then, all the databases are populated by means of the SQL statements produced by single `pg_dump` runs.

`pg_dumpall` only produces an SQL script, so you need to restore your cluster by means of `psql` or an interactive connection. All the main options you can use with `pg_dump` that have been presented in the previous sections apply to `pg_dumpall` too.

`pg_dumpall` provides a particularly useful option, `--globals-only`, which is used to dump only intra-cluster objects, such as roles, tablespaces, and replication slots. This option is useful to dump and restore such objects across different clusters:

```
$ pg_dumpall --globals-only -f cluster.sql
```

Parallel backups

It is possible to use parallelization to speed up backups and restores. The basic idea is to have multiple processes (and database connections), each assigned a smaller task to perform, so that performing all the tasks in parallel will provide you with better performance.

It is important to note that, often, it is not doing the backup faster that's the problem – rather, it's being able to perform the restoration as fast as you can. So, while it is possible to perform both backups and restoration in parallel mode, you will find restoration to be the most important one.

`pg_dump` allows you to specify the parallelism level via the `-j` command-line argument, to which you must assign a positive integer – that is, the number of parallel processes to start. `pg_dump` will then open parallel connections to the database in number equal to the parallelism, plus one connection to rule them all, and will force every connection to dump a separate table. This clearly means it does not make any sense to start more processes than the number of tables in your database that you need to back up.

Since all the processes will dump a single table, parallel mode is only available for the directory (`-Fd`) format, where every table is stored in a separate file so that processes don't mix their writes together.

As an example, the following instruction will dump the database with three parallel jobs, thus opening four database connections:

```
$ pg_dump -Fd -f backup_forumdb -v -j 3 forumdb
...
pg_dump: finished item 3373 TABLE DATA posts
pg_dump: finished item 3370 TABLE DATA delete_posts
pg_dump: finished item 3377 TABLE DATA users
pg_dump: dumping contents of table "forum.j_posts_tags"
pg_dump: dumping contents of table "forum.new_categories"
pg_dump: dumping contents of table "forum.tags"
pg_dump: finished item 3372 TABLE DATA new_categories
```

```
pg_dump: finished item 3375 TABLE DATA tags
pg_dump: finished item 3371 TABLE DATA j_posts_tags
```

Messages such as `finished item` are the single dumping processes that are completed as a single table, and they will not be shown in the non-parallel verbose output of the `pg_dump` command. It is important to consider the number of connections opened by a parallel `pg_dump`: they are always done on every parallel job, plus one, to synchronize and manage the whole backup procedure. This means that in order to execute a parallel backup, you must ensure there are enough connections available against your database; otherwise, the backup will fail.

Another important aspect of parallel backups is that they could fail under concurrent circumstances. In fact, once `pg_dump` has started, the “master” process acquires light locks (shared locks) on every object the parallel processes will dump, while, when started, every parallel process acquires an exclusive (heavy) lock on the object. This prevents the object (a table) from being destroyed before the parallel process has finished doing its work. However, between the acquisition of the first lock from the master process and the acquisition of the heavy lock from its spawned parallel process, another concurrent connection could try to acquire the lock on the table, resulting in a possible deadlock situation. To prevent this, the master `pg_dump` process will detect the dependency and abort the whole backup.

`pg_restore` does support parallel restoration too, by means of the same mnemonic `-j` command-line argument. The command will spawn the indicated number of processes involved in data loading, index creation, and all the other heavy and time-consuming operations.

Unlike `pg_dump`, `pg_restore` can work in parallel for both the directory format and the custom format. It is not simple to determine the number of parallel jobs to specify to `pg_restore`, but usually, this is the number of CPU cores, even if values slightly greater than that can produce a faster restoration.

As an example, the following command allows for parallel restoration of the backup we took previously (the first line drops the database for the restoration to succeed):

```
$ psql -c "DROP DATABASE forumdb;"
$ pg_restore -C -d postgres -j 4 -v backup.d
...
pg_restore: finished item 3388 SEQUENCE SET users_pk_seq
pg_restore: finished item 3387 SEQUENCE SET tags_pk_seq
```